

Verilog 第二次实验报告

|      |            |    |         |            |     |
|------|------------|----|---------|------------|-----|
| 实验名称 | 作业二        |    |         |            |     |
| 学生姓名 | 张耕嘉        | 学号 | 2313725 | 指导老师       | 董前琨 |
| 实验地点 | 综合实验楼 A306 |    | 实验时间    | 2024.11.14 |     |

- 一、实验项目名称实验二
- 二、实验目的熟练掌握 Verilog 中语句的使用；熟悉使用 Vivado 编写 Verilog 程序的流程
- 三、必修或选修必修
- 四、实验平台Vivado
- 五、实验内容及步骤

1. always 语句和 assign 语句的区别

always 语句可以描述组合逻辑，也可以描述时序逻辑。用于描述时序逻辑时，其中包含了触发器（寄存器）的行为，可以响应时钟信号或者特定的触发条件。always 语句中的代码块会在满足触发条件时执行，因此描述了一种在时间上演进的行为；assign 语句用于连续赋值。它只能描述组合逻辑，其中输出信号的值由输入信号的值直接确定。

2. 变量 reg 和 wire 的区别

首先,reg 型变量可以生成组合逻辑，也可以生成触发器，而 wire 型变量只能生成组合逻辑。其次，reg 型变量只能通过过程赋值语句赋值，不能通过 assign 语句赋值，而 wire 型变量可以通过 assign 语句赋值，但不能在过程块中被赋值。最后，reg 型可以用来存储状态，即 reg 被再次赋值前会保持原值，wire 型变量没有这个功能。

3. 真值表

| a[0] | a[1] | b[0] | b[1] | equal |
|------|------|------|------|-------|
| 0    | 0    | 0    | 0    | 1     |
| 0    | 0    | 0    | 1    | 0     |
| 0    | 0    | 1    | 0    | 0     |
| 0    | 0    | 1    | 1    | 0     |
| 0    | 1    | 0    | 0    | 0     |
| 0    | 1    | 0    | 1    | 1     |
| 0    | 1    | 1    | 0    | 0     |
| 0    | 1    | 1    | 1    | 0     |
| 1    | 0    | 0    | 0    | 0     |
| 1    | 0    | 0    | 1    | 0     |
| 1    | 0    | 1    | 0    | 1     |
| 1    | 0    | 1    | 1    | 0     |
| 1    | 1    | 0    | 0    | 0     |
| 1    | 1    | 0    | 1    | 0     |
| 1    | 1    | 1    | 0    | 1     |
| 1    | 1    | 1    | 1    | 1     |

1. 半加器

编写设计模块

```
module halfadder(  
    input a,  
    input b,  
    output sum,  
    output c  
);  
    assign sum=a^b;  
    assign c=a&b;  
endmodule
```

编写仿真模块

```
module testbench();  
    reg tempa,tempb;//输入定义为 reg  
    wire tsum,tc;//输出定义为 wire  
    halfadder myha(tempa,tempb,tsum,tc);  
    initial begin  
        tempa=1'b0;  
        tempb=1'b0;  
    end  
    always #5 tempa=~tempa;  
    always #3 tempb=~tempb;  
endmodule
```

2. 全加器

编写设计模块

编写仿真模块

```

module fulladd(
    input a,
    input b,
    input ci,
    output sum,
    output co
);
    wire t1,t2,t3;
    halfadder myha1(a,b,t1,t2);
    halfadder myad2(t1,ci,sum,t3);
    assign co=t2^t3;
endmodule

```

```

module testbench();
    reg tempaa,tempbb,tempcii;
    wire tempsum,tempco;
    initial begin
        tempaa=1'b0;
        tempbb=1'b0;
        tempcii=1'b0;
    end
    fulladd myfa(tempaa,tempbb,tempcii,tempsum,tempco);
    always #5 tempaa=~tempaa;
    always #3 tempbb=~tempbb;
    always #7 tempcii=~tempcii;
endmodule

```

### 3.八位加法器 编写设计模块

### 编写仿真模块

```

module addeight(input [7:0] ain,input [7:0] bin,input cin,output [7:0] sumout,output
cout);
    wire [7:0] carry;
    fulladd fad0(ain[0], bin[0], cin, sumout[0], carry[0]);
    fulladd fad1(ain[1], bin[1], carry[0], sumout[1], carry[1]);
    fulladd fad2(ain[2], bin[2], carry[1], sumout[2], carry[2]);
    fulladd fad3(ain[3], bin[3], carry[2], sumout[3], carry[3]);
    fulladd fad4(ain[4], bin[4], carry[3], sumout[4], carry[4]);
    fulladd fad5(ain[5], bin[5], carry[4], sumout[5], carry[5]);
    fulladd fad6(ain[6], bin[6], carry[5], sumout[6], carry[6]);
    fulladd fad7(ain[7], bin[7], carry[6], sumout[7], carry[7]);
    assign cout=carry[7];
endmodule

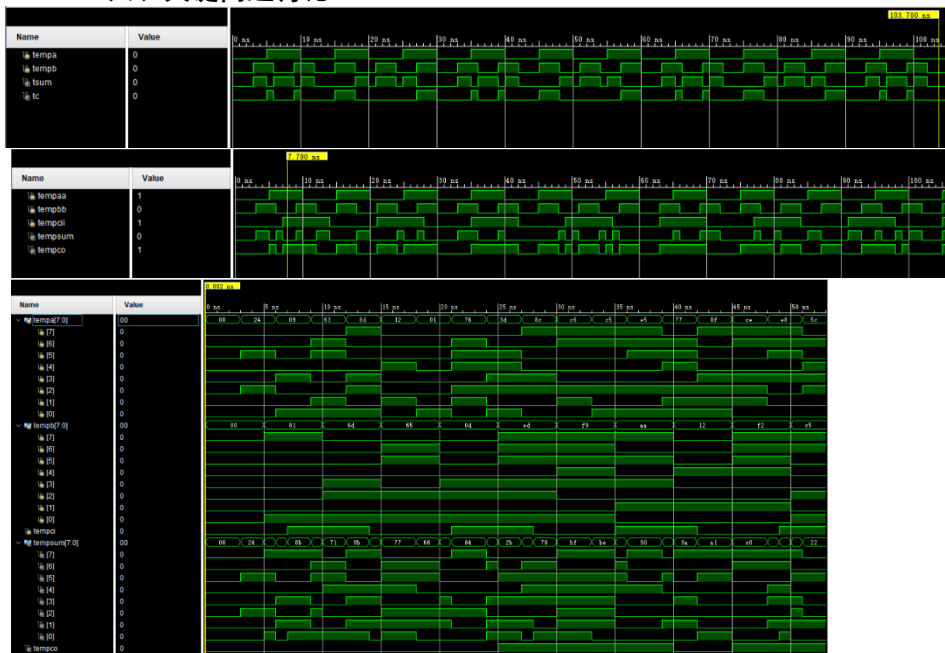
```

```

module testbench();
    reg [7:0] tempa,tempb;
    reg tempci;
    wire [7:0] tempsum;
    wire tempco;
    addeight ea(tempa,tempb,tempci,tempsum,tempco);
    initial begin
        tempa=8'b0000_0000;
        tempb=8'b0000_0000;
        tempci=1'b0;
    end
    always #3 tempa = $random % 9'b1_0000_0000;
    always #5 tempb = $random % 9'b1_0000_0000;
    always #7 tempci=~tempci;
endmodule

```

## 六、关键问题讨论



如上图所示，经过仿真测试，所有的输出结果都符合要求（这里只截取了部分测试数据）。

## 七、总结

通过这次实验，我掌握了 Verilog 中各种运算符的原理及使用方法，也更加熟悉了如何使用 Vivado 编写程序。